OxigenNative

Compilador de OxigenScript a ARM64

Manual de Usuario

Organización de Lenguajes y Compiladores 2 — Proyecto 2

Guía del IDE OxigenNative: instalación, ejecución en ARM64 con QEMU, reportes de errores, tabla de símbolos, AST y tokens, y referencia del lenguaje.

Autor: Angel Raúl Herrera Chilel
Carnet: 202402587
Sección: B

Fecha: 23 de octubre de 2026

Índice

Introducción **3**

1 Requerimientos e Instalación **4**
1.1 Obtener el código . . . 4
1.2 Instalar las dependencias del sistema . . . 4
1.3 Instalar las dependencias de Python . . . 4
1.4 Instalar las dependencias del IDE web . . . 4
1.5 Verificar la cadena ARM64 . . . 5

2 Iniciar y detener la aplicación **5**

3 La interfaz **6**
3.1 Atajos de teclado . . . 6
3.2 Tema claro y oscuro . . . 7

4 Crear, abrir, editar y guardar **7**

5 Ejecutar un programa **7**
5.1 Cuándo se ejecuta con el intérprete aunque el modo sea ARM64 . . . 8
5.2 El panel «Código ARM64 generado» . . . 9

6 Reportes **9**
6.1 Errores . . . 10
6.2 Tabla de símbolos . . . 11
6.3 AST . . . 13
6.4 Tokens . . . 14

7 Sesiones de uso **14**
7.1 Programa correcto en ARM64 . . . 14
7.2 Encontrar y corregir errores . . . 15
7.3 Errores de ejecución sin detener el programa . . . 16
7.4 Error semántico en modo ARM64 . . . 17

8 Manual del Lenguaje **17**
8.1 Variables y tipos . . . 17
8.1.1 Sintaxis . . . 17
8.1.2 Tipos de dato . . . 17
8.1.3 Valores por defecto . . . 18
8.1.4 Ejemplo . . . 18
8.1.5 Operadores de asignación . . . 18
8.2 Operadores . . . 18
8.2.1 Aritméticos . . . 19
8.2.2 Relacionales . . . 19
8.2.3 Lógicos . . . 19
8.3 Control de flujo . . . 19
8.3.1 Condicionales . . . 19
8.3.2 Bucles . . . 20
8.3.3 Bucles con etiquetas . . . 20
8.3.4 Match . . . 21
8.4 Funciones . . . 21

8.5 Arreglos 21
8.6 Strings 22
8.7 Structs y slices 22
8.8 Funciones embebidas 23
8.9 Comentarios 23

9 Solución de Problemas **24**

10 Conclusiones y Recomendaciones **24**
10.1 Conclusiones 24
10.2 Recomendaciones 24

Introducción

OxigenNative es un **compilador** para OxigenScript, un lenguaje con sintaxis inspirada en Rust. Analiza el código fuente, construye un árbol de sintaxis abstracta (AST), lo valida y lo traduce a **ensamblador ARM64 (AArch64)**. El ensamblador se ensambla y enlaza con las herramientas GNU para AArch64 y se ejecuta con **QEMU User Mode**.

La herramienta se usa desde un **IDE web**: se escribe el programa, se ejecuta y se revisan, en la misma pantalla, la salida, el código ARM64 generado y los reportes de errores, de tabla de símbolos, del AST y de tokens. El IDE también conserva el **modo intérprete** de la primera fase, que ejecuta el AST directamente en Python.

Este manual está dirigido a quien usa el sistema. No se necesitan conocimientos de compiladores ni de ensamblador: basta conocer la sintaxis del lenguaje, que se resume en la sección 8, y saber operar la interfaz web.

Requerimientos e Instalación

Componente	Versión	Para qué
Linux	reciente	Sistema operativo del servidor.
Python	3.14 o superior	Compilador y API.
uv	reciente	Entorno virtual y dependencias de Python.
Node.js	**22.12 o superior**	IDE web. Astro 7 no arranca con versiones menores.
Graphviz	reciente	Imagen del AST (comando `dot`).
binutils para AArch64	2.40 o superior	`aarch64-linux-gnu-as`, `-ld` y `-ar`.
QEMU User Mode	8 o superior	`qemu-aarch64`, que ejecuta el binario ARM64.
Navegador	Firefox, Chrome	Usar el IDE.

El proyecto se probó con Python 3.14.8, Node 24.18, GNU Binutils 2.47 y QEMU 11.1.

Obtener el código

```
git clone https://github.com/Angelius6573/OLC2_B_P2_2S2026.git
   OxigenNative
cd OxigenNative
```

Instalar las dependencias del sistema

Arch Linux y derivadas (Manjaro, EndeavourOS, CachyOS):

```
sudo pacman -S python uv graphviz nodejs npm aarch64-linux-gnu-binutils
   qemu-user
```

Debian, Ubuntu y derivadas (Linux Mint, Pop!_OS, Zorin OS):

```
sudo apt update
sudo apt install python3 graphviz nodejs npm binutils-aarch64-linux-gnu
   qemu-user
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Si la distribución no ofrece Python 3.14, `uv` lo descarga en el paso siguiente.

Instalar las dependencias de Python

```
uv sync
```

Esto crea la carpeta `.venv/` con PLY, FastAPI, Uvicorn y la biblioteca de Graphviz.

Instalar las dependencias del IDE web

Primero hay que comprobar la versión de Node:

```
node --version          # debe ser v22.12 o mayor
```

Si es menor, se puede instalar una versión nueva con `nvm`:

```
nvm install 24
nvm use 24
```

Luego se instalan las dependencias del frontend. Este paso es obligatorio en un clon nuevo: sin él, el IDE no arranca.

```
cd frontend
npm install
cd ..
```

Verificar la cadena ARM64

```
aarch64-linux-gnu-as --version | head -1
aarch64-linux-gnu-ld --version | head -1
qemu-aarch64 --version | head -1
dot -V
```

Los cuatro comandos deben responder con su versión. Si falta alguna herramienta, el IDE lo indica al ejecutar en modo ARM64 con el mensaje «Faltan herramientas: ...».

Iniciar y detener la aplicación

Desde la raíz del proyecto:

```
uv run start.sh
```

El script levanta dos servidores y escribe sus registros en `/tmp`:

Servidor	Dirección	Registro
API (FastAPI)	`http://localhost:8000`	`/tmp/fastapi.log`
IDE web (Astro)	`http://localhost:4321`	`/tmp/astro.log`

Abra `http://localhost:4321` en el navegador; la página redirige al IDE. Para detener ambos servidores, presione `Ctrl+C` en la terminal del script.

> El script muestra «Servidores listos» cuando los dos procesos siguen vivos a los pocos segundos de arrancar. Si el IDE no abre aunque aparezca ese mensaje, revise `/tmp/astro.log`: la causa más común es una versión de Node menor que 22.12 o no haber ejecutado `npm install` en `frontend/`.

La interfaz

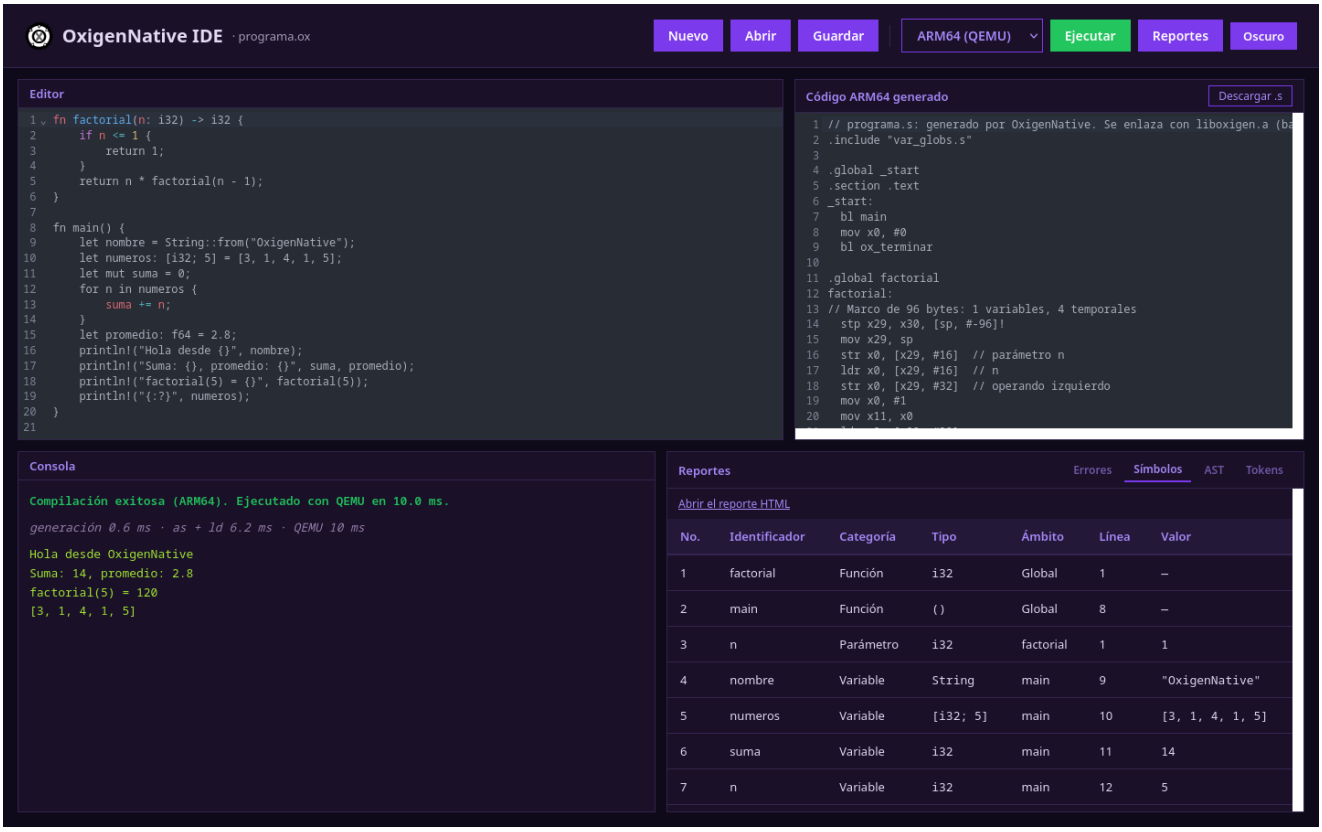


Figura 1: Vista general del IDE con un programa ejecutado en ARM64

La pantalla se divide en cinco áreas:

Área	Ubicación	Contenido
Barra de herramientas	arriba	Nuevo, Abrir, Guardar, selector de modo, Ejecutar, Reportes y tema claro u oscuro. A la izquierda, el nombre del archivo abierto.
Editor	centro izquierda	El código fuente, con números de línea, resaltado de sintaxis y cierre automático de paréntesis.
Código ARM64 generado	centro derecha	El `programa.s` que se ensambló y ejecutó en QEMU, con números de línea y el botón «Descargar .s».
Consola	abajo izquierda	Mensaje de estado, tiempos de cada etapa, salida del programa y errores.
Reportes	abajo derecha	Pestañas Errores, Símbolos, AST y Tokens.

Atajos de teclado

Atajo	Acción
`Ctrl + Enter`	Ejecutar el programa
`Ctrl + Z` / `Ctrl + Y`	Deshacer / Rehacer
`Ctrl + F`	Buscar en el editor

Tema claro y oscuro

El botón de la esquina superior derecha cambia entre el tema oscuro y el claro, también en los dos editores. La preferencia se recuerda al volver a abrir el IDE.

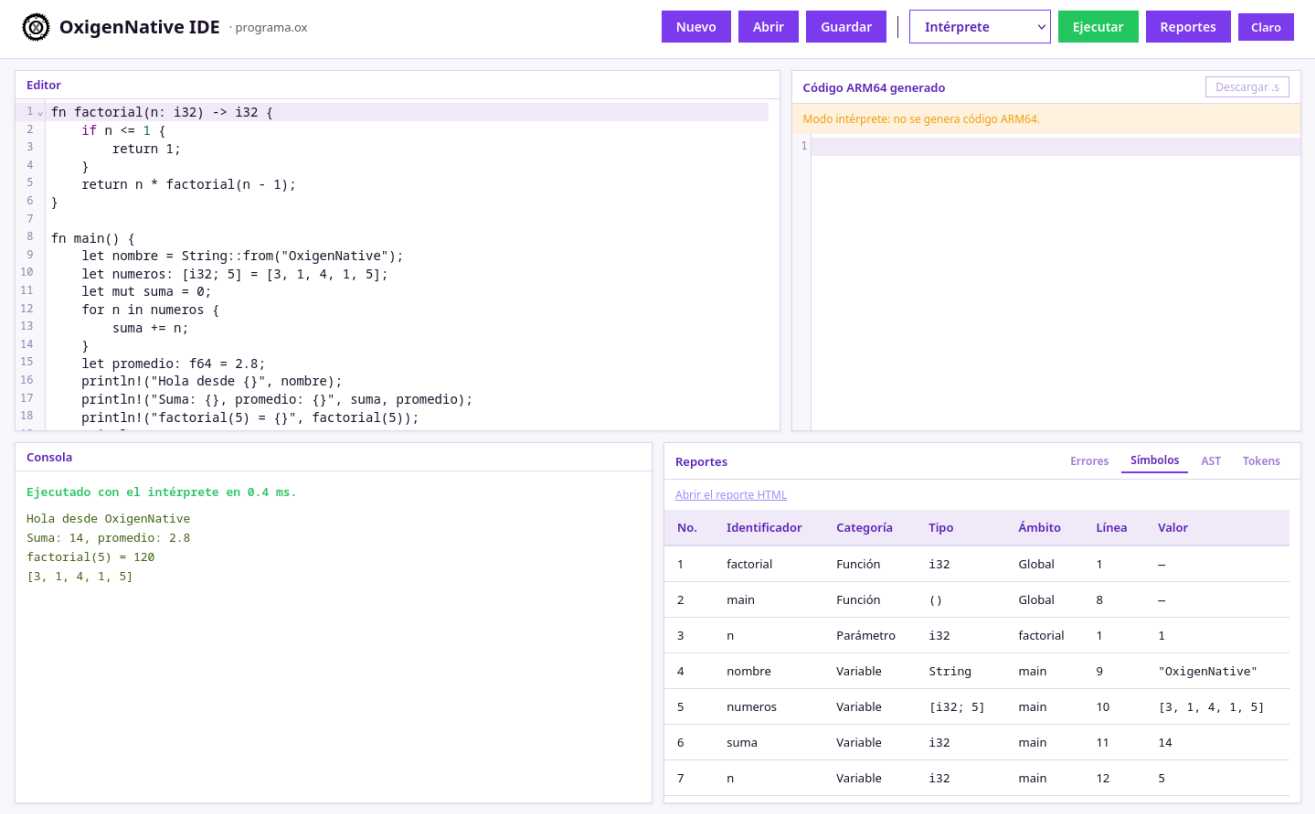


Figura 2: Tema claro, con el selector en modo Intérprete

Crear, abrir, editar y guardar

Botón	Función
Nuevo	Vacía el editor y los reportes. Si hay código escrito, pide confirmación. El archivo se llama `programa.ox`.
Abrir	Muestra el selector de archivos del sistema. Al elegir un `.ox` (o `.txt`), su contenido se carga en el editor y su nombre aparece en la barra. En `docs/manual/ejemplos/` hay programas listos para abrir.
Guardar	Descarga el código del editor como un archivo con el nombre actual, por ejemplo `programa.ox`, en la carpeta de descargas del navegador.

El editor guarda el código en el navegador mientras se escribe: si se recarga la página, el último código sigue ahí.

Ejecutar un programa

1. Elija el modo en el selector de la barra:
   - **ARM64 (QEMU)**, el modo por omisión: compila a ARM64, ensambla, enlaza y ejecuta con QEMU.

- **Intérprete**: ejecuta el AST directamente en Python, sin generar ensamblador.

2. Presione **Ejecutar** (o `Ctrl+Enter`).
3. Lea el resultado en la consola:
   - el **mensaje de estado**, en verde si todo salió bien o en rojo si hubo errores; por ejemplo, «Compilación exitosa (ARM64). Ejecutado con QEMU en 10.0 ms.»;
   - los **tiempos** de cada etapa: generación, `as` + `ld` y QEMU;
   - la **salida** del programa, en verde;
   - los **errores**, en rojo, con el formato del enunciado.

   Si el programa tiene errores, la pestaña **Errores** se abre sola.

Cuándo se ejecuta con el intérprete aunque el modo sea ARM64

El aviso naranja del panel ARM64 explica cada uno de estos casos:

Situación	Qué pasa
Errores léxicos o sintácticos	No se ejecuta: «No se generó código: el programa tiene errores.»
Errores semánticos	No se genera ARM64. El programa se ejecuta con el intérprete para mostrar lo que sí funciona.
Structs o slices	El generador ARM64 no los soporta; se ejecuta con el intérprete.

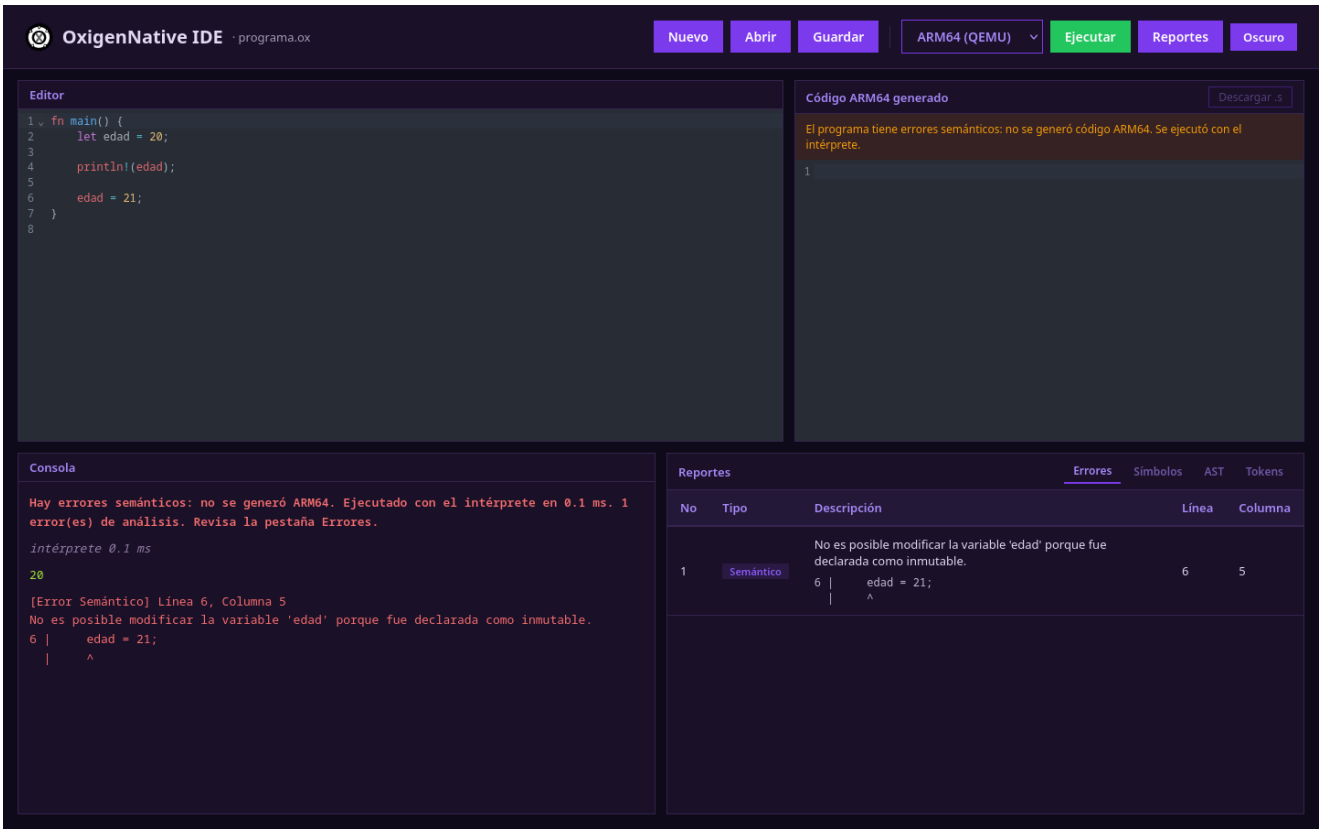


Figura 3: Un error semántico en modo ARM64: el programa se ejecuta con el intérprete y el panel lo avisa

El panel «Código ARM64 generado»

Muestra exactamente el `programa.s` que se ejecutó. Cada función empieza con su etiqueta (`main:`), un comentario con el tamaño de su marco de pila y el prólogo (`stp x29, x30, ...`). Cada `let` lleva un comentario con el lugar de la variable en el marco, por ejemplo `// let x: i32 ->[x29, #16]`.

El botón **Descargar .s** guarda el archivo para ensamblarlo a mano desde la raíz del proyecto:

```
aarch64-linux-gnu-as -I backend/arm64/lib -o programa.o programa.s
aarch64-linux-gnu-ld -o programa programa.o build/arm64/liboxigen.a
qemu-aarch64 ./programa
```

Reportes

Los reportes están en las pestañas del panel inferior derecho. El botón **Reportes** de la barra lleva a cada pestaña y abre el reporte HTML de símbolos. Todos se generan de nuevo en cada ejecución.

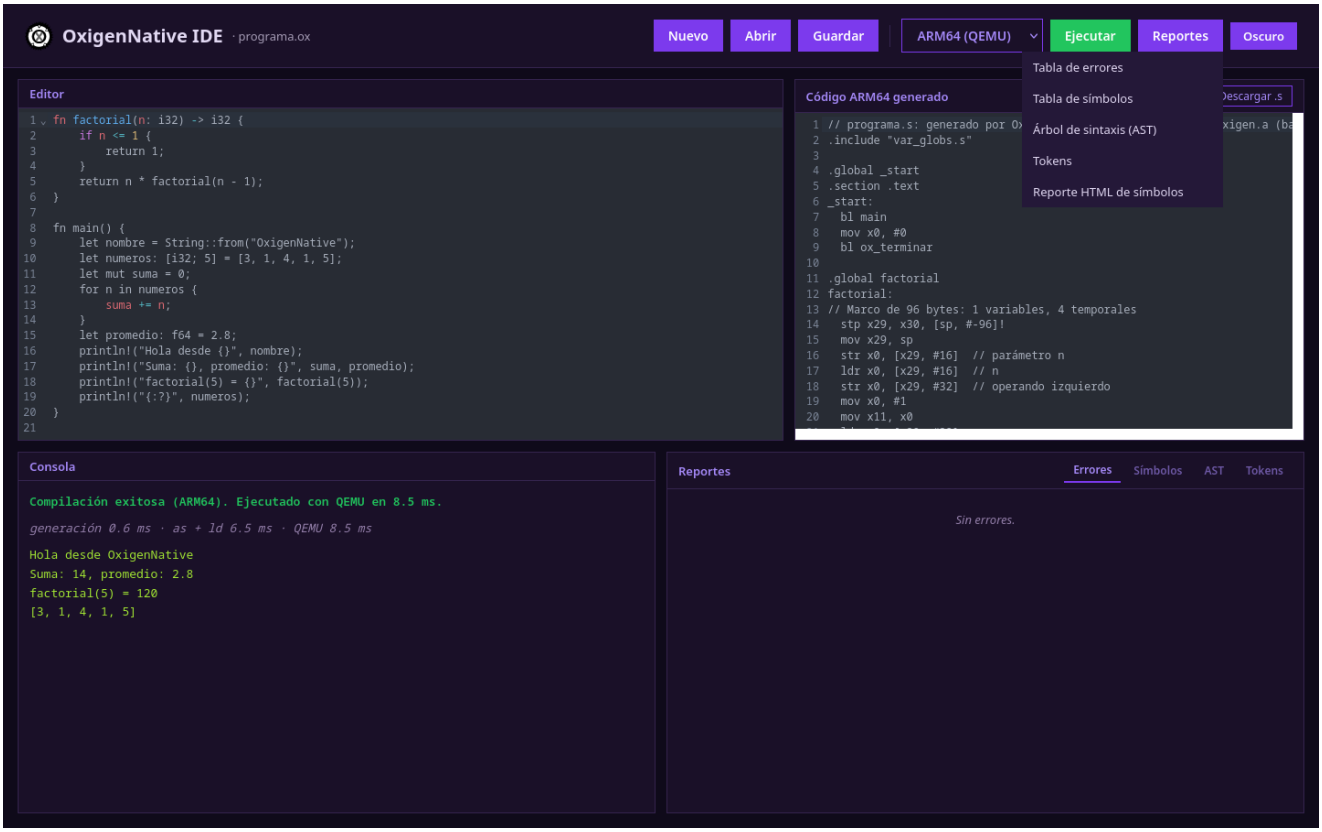


Figura 4: Menú Reportes de la barra de herramientas

Errores

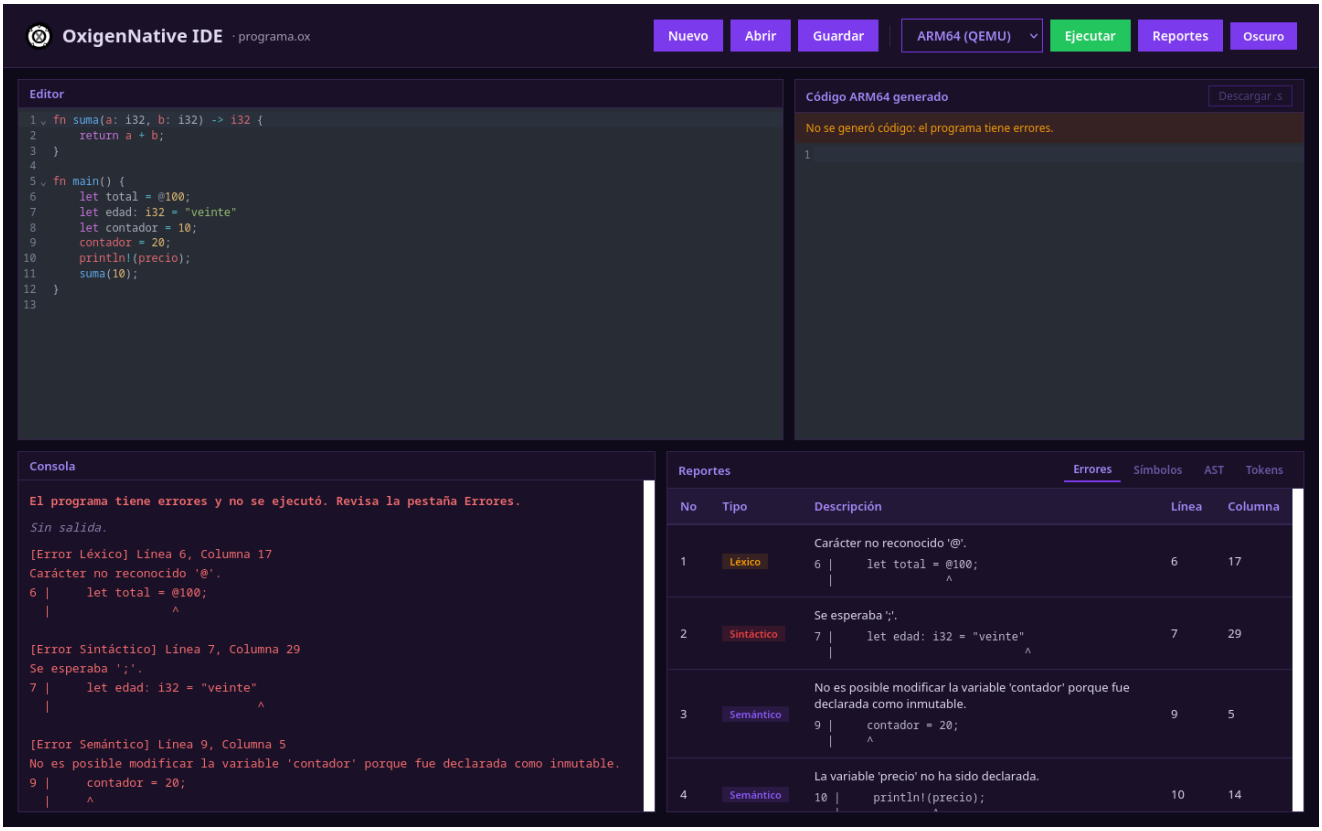


Figura 5: Tabla de errores con errores léxicos, sintácticos y semánticos

Es la tabla de análisis de errores. Cada fila tiene:

Columna	Significado
No	Número del error en la ejecución.
Tipo	Léxico, Sintáctico, Semántico, Ejecución o Generación, cada uno con su color.
Descripción	Qué está mal y, debajo, la línea del código con un ˆ en la columna exacta.
Línea / Columna	Posición del error, contando desde 1.

Un clic en una fila lleva el cursor del editor a esa línea y columna.

Los errores aparecen ordenados por posición, y en una sola ejecución pueden aparecer errores de las tres fases de análisis:

Tipo	Cuándo aparece	Ejemplo
Léxico	Un símbolo que no pertenece al lenguaje, o un `/*` sin `*/`	Carácter no reconocido '@'.
Sintáctico	El código no sigue la gramática	Se esperaba ';'. La columna señala justo después del último token de la línea.
Semántico	El código está bien escrito, pero no tiene sentido	La variable 'precio' no ha sido declarada.
Ejecución	Falla al correr	División por cero. El programa **sigue** con la siguiente sentencia.
Generación	Falla el ensamblado o el enlace	Muestra la línea de `programa.s` y la instrucción.

En la consola, los mismos errores aparecen como texto:

```
[Error Semántico] Línea 10, Columna 14
La variable 'precio' no ha sido declarada.
10 |     println!(precio);
   |               ^
```

Tabla de símbolos

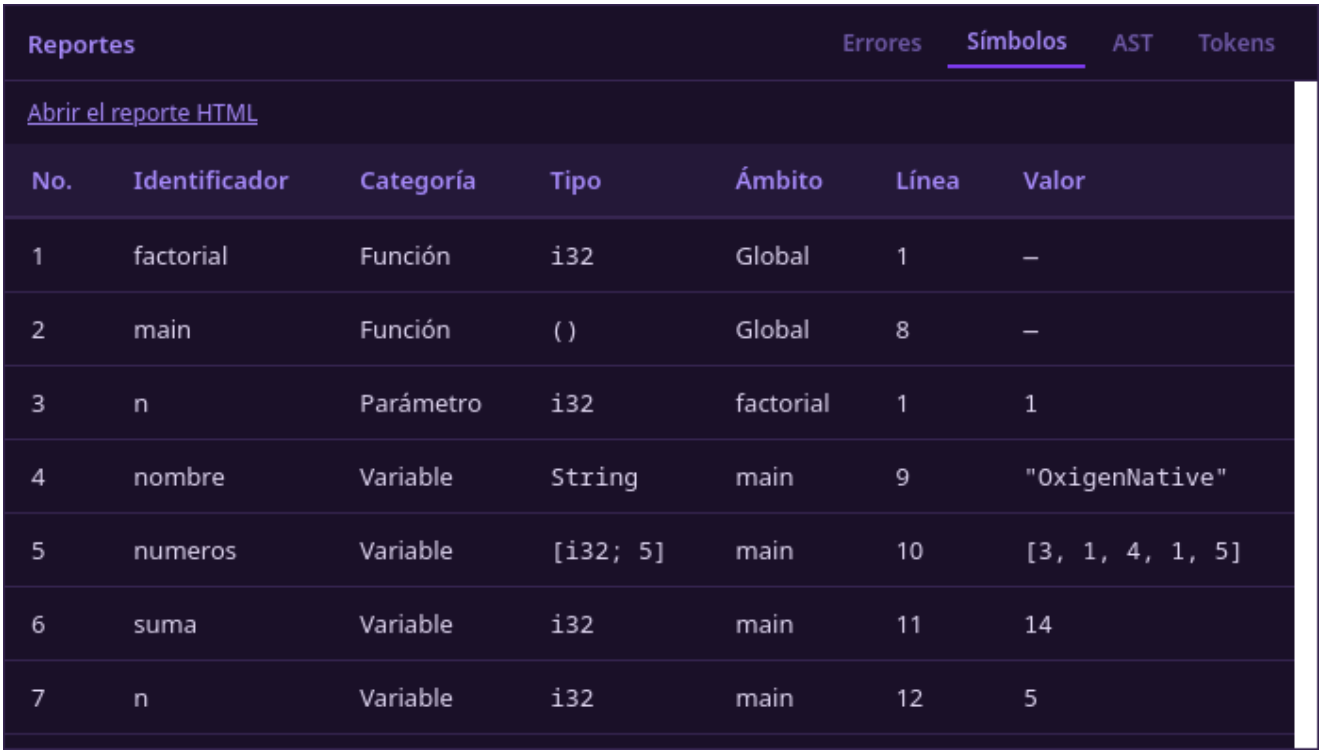


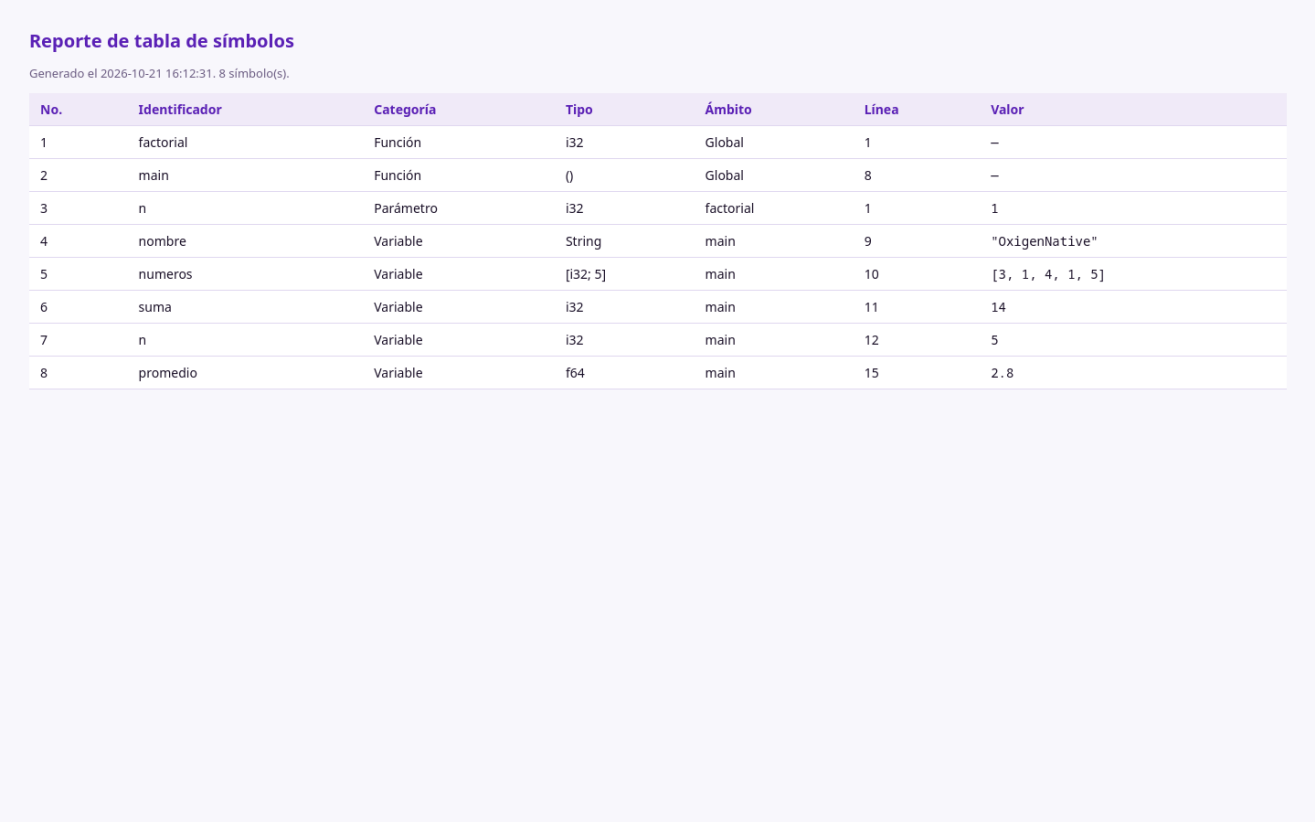
Figura 6: Pestaña Símbolos

Lista cada identificador registrado durante el análisis:

Columna	Significado
No.	Orden de registro: primero funciones y structs, luego variables.
Identificador	Nombre declarado.
Categoría	Función, Struct, Variable o Parámetro.
Tipo	`i32`, `f64`, `bool`, `char`, `String`, `[i32; 5]`... En una función, su tipo de retorno; `()` si no devuelve nada.
Ámbito	`Global` para funciones y structs; el nombre de la función para variables y parámetros.
Línea	Línea de la declaración.
Valor	El último valor que tuvo la variable al terminar la ejecución; si nunca se asignó, el valor por defecto de su tipo. Las funciones muestran «—».

Una variable que se declara dos veces con el mismo nombre (*shadowing*) aparece dos veces, porque son dos símbolos distintos. Un parámetro de una función recursiva muestra el valor de la última llamada.

El enlace **«Abrir el reporte HTML»** (o Reportes → «Reporte HTML de símbolos») abre el reporte en una pestaña nueva del navegador:



Reporte de tabla de símbolos

Generado el 2026-10-21 16:12:31. 8 símbolo(s).

No.	Identificador	Categoría	Tipo	Ámbito	Línea	Valor
1	factorial	Función	i32	Global	1	–
2	main	Función	()	Global	8	–
3	n	Parámetro	i32	factorial	1	1
4	nombre	Variable	String	main	9	"OxigenNative"
5	numeros	Variable	[i32; 5]	main	10	[3, 1, 4, 1, 5]
6	suma	Variable	i32	main	11	14
7	n	Variable	i32	main	12	5
8	promedio	Variable	f64	main	15	2.8

Figura 7: Reporte HTML de la tabla de símbolos

AST

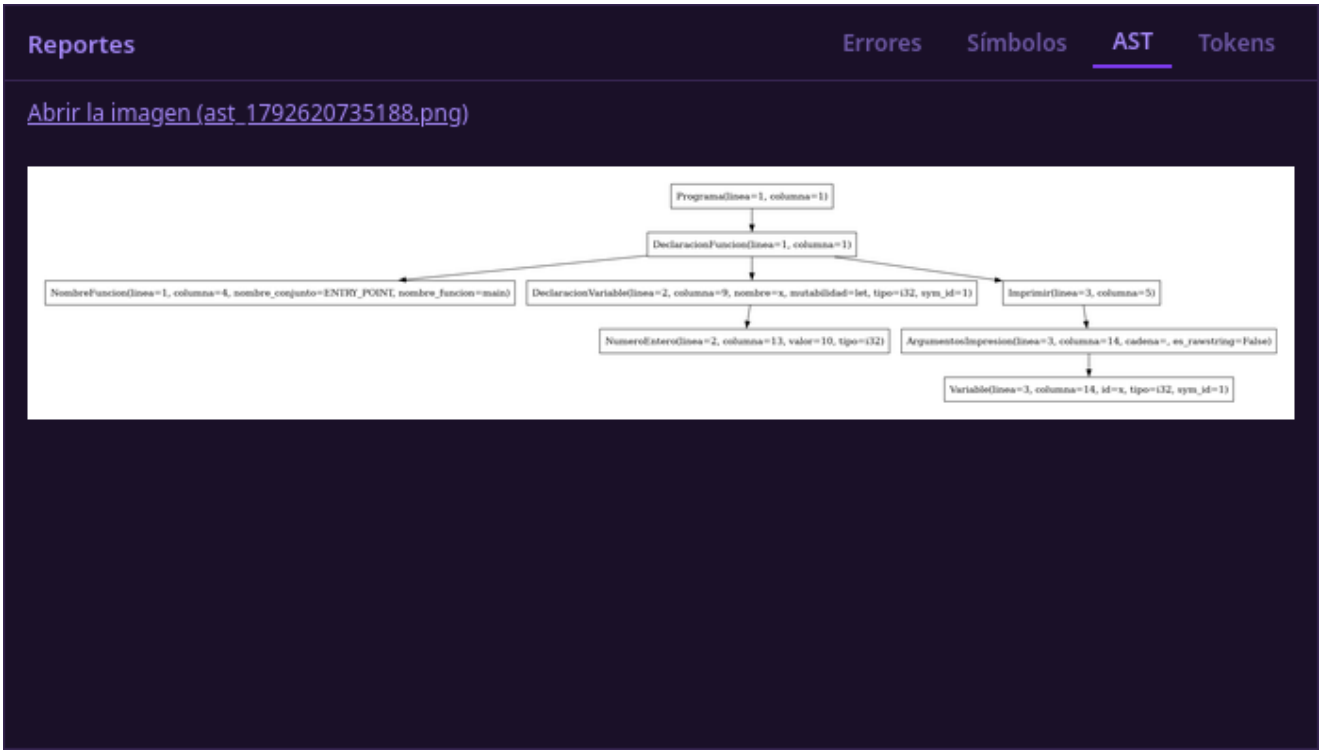


Figura 8: Pestaña AST

Muestra el árbol de sintaxis abstracta generado con Graphviz. La raíz es `Programa`. Cada nodo indica su clase y sus datos: línea, columna, nombre, valor y, después del análisis, su `tipo` y su `sym_id`. «Abrir la imagen» la muestra a tamaño completo, lo que es útil en programas largos.

Tokens

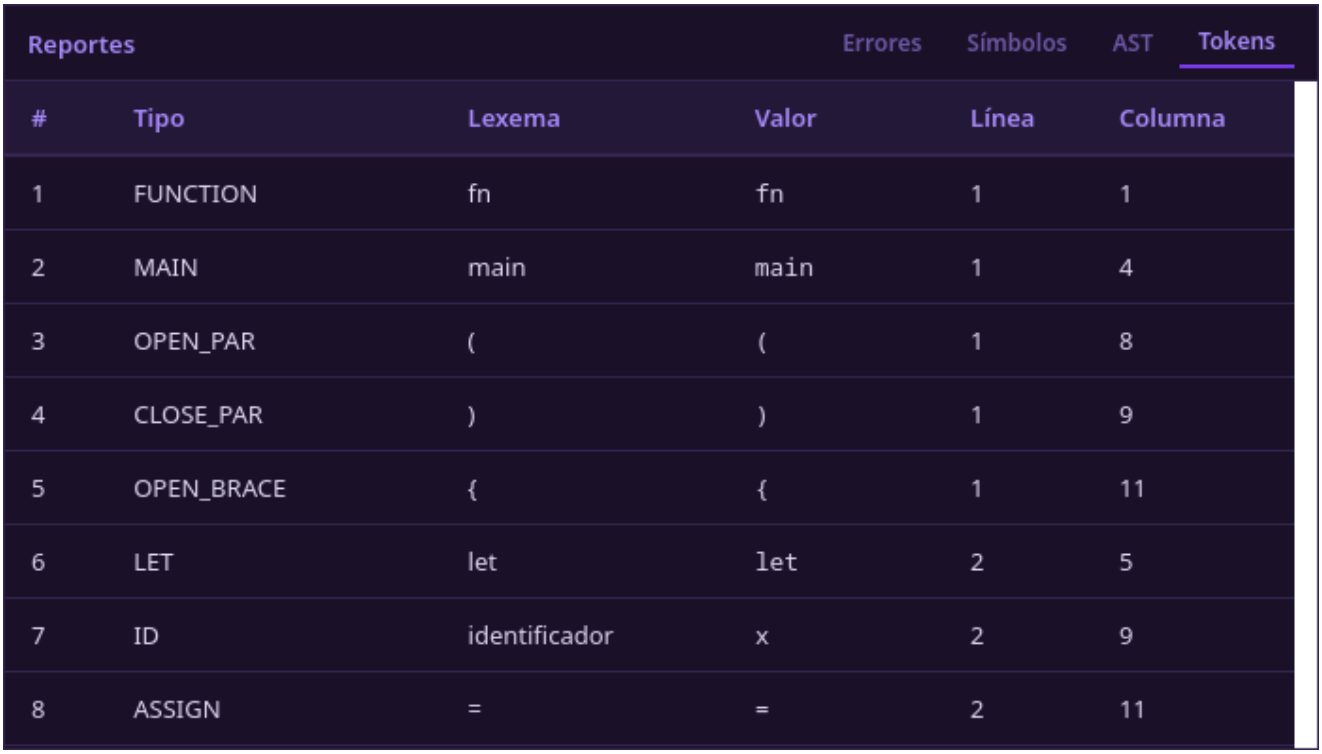


Figura 9: Pestaña Tokens

Lista los tokens que reconoce el analizador léxico, con su tipo, lexema, valor, línea y columna. Sirve para ver cómo se separa el código antes del análisis sintáctico.

Sesiones de uso

Los cuatro programas de esta sección están en `docs/manual/ejemplos/` y se abren con el botón **Abrir**.

Programa correcto en ARM64

Archivo `01_ejemplo.ox`:

Listing 1: 01_ejemplo.ox

```
fn factorial(n: i32) -> i32 {
    if n <= 1 {
        return 1;
    }
    return n * factorial(n - 1);
}

fn main() {
    let nombre = String::from("OxigenNative");
    let numeros: [i32; 5] = [3, 1, 4, 1, 5];
    let mut suma = 0;
    for n in numeros {
```

```
        suma += n;
    }
    let promedio: f64 = 2.8;
    println!("Hola desde {}", nombre);
    println!("Suma: {}, promedio: {}", suma, promedio);
    println!("factorial(5) = {}", factorial(5));
    println!("{:?}", numeros);
}
```

Con el modo **ARM64 (QEMU)**, presione **Ejecutar**. La consola muestra:

```
Compilación exitosa (ARM64). Ejecutado con QEMU en 10.0 ms.
Hola desde OxigenNative
Suma: 14, promedio: 2.8
factorial(5) = 120
[3, 1, 4, 1, 5]
```

El panel ARM64 tiene el código de `factorial` y `main`, y la pestaña Símbolos muestra `suma = 14` y `nombre = .ºxigenNative"` (figura de la vista general, en la sección de la interfaz).

Encontrar y corregir errores

Archivo `02_errores.ox`:

Listing 2: 02_errores.ox

```
fn suma(a: i32, b: i32) -> i32 {
    return a + b;
}

fn main() {
    let total = @100;
    let edad: i32 = "veinte"
    let contador = 10;
    contador = 20;
    println!(precio);
    suma(10);
}
```

Al ejecutar, la tabla de errores muestra cinco errores de las tres fases de análisis:

No	Tipo	Descripción	Línea	Columna
1	Léxico	Carácter no reconocido '@'.	6	17
2	Sintáctico	Se esperaba ';'.	7	29
3	Semántico	No es posible modificar la variable 'contador' porque fue declarada como inmutable.	9	5
4	Semántico	La variable 'precio' no ha sido declarada.	10	14
5	Semántico	La función 'suma' esperaba 2 argumentos, pero recibió 1.	11	5

Para corregirlos, haga clic en cada fila para ir a la línea y:

1. quite el `@` (`let total = 100;`);
2. agregue el `;` al final de la línea 7 y use un número (`let edad: i32 = 20;`);
3. declare `contador` con `let mut`;

4. declare `precio` antes de usarlo, por ejemplo `let precio = 15;`;
5. pase los dos argumentos: `suma(10, 5);`.

Al ejecutar otra vez, la consola muestra «Compilación exitosa (ARM64)» y la tabla dice «Sin errores».

Errores de ejecución sin detener el programa

Archivo `03_error_ejecucion.ox`:

Listing 3: 03_error_ejecucion.ox

```
fn main() {
    let datos = [10, 20, 30];
    let divisor = 0;
    println!("Inicio");
    println!("{}", 100 / divisor);
    let i = 5;
    println!("{}", datos[i]);
    println!("Fin");
}
```

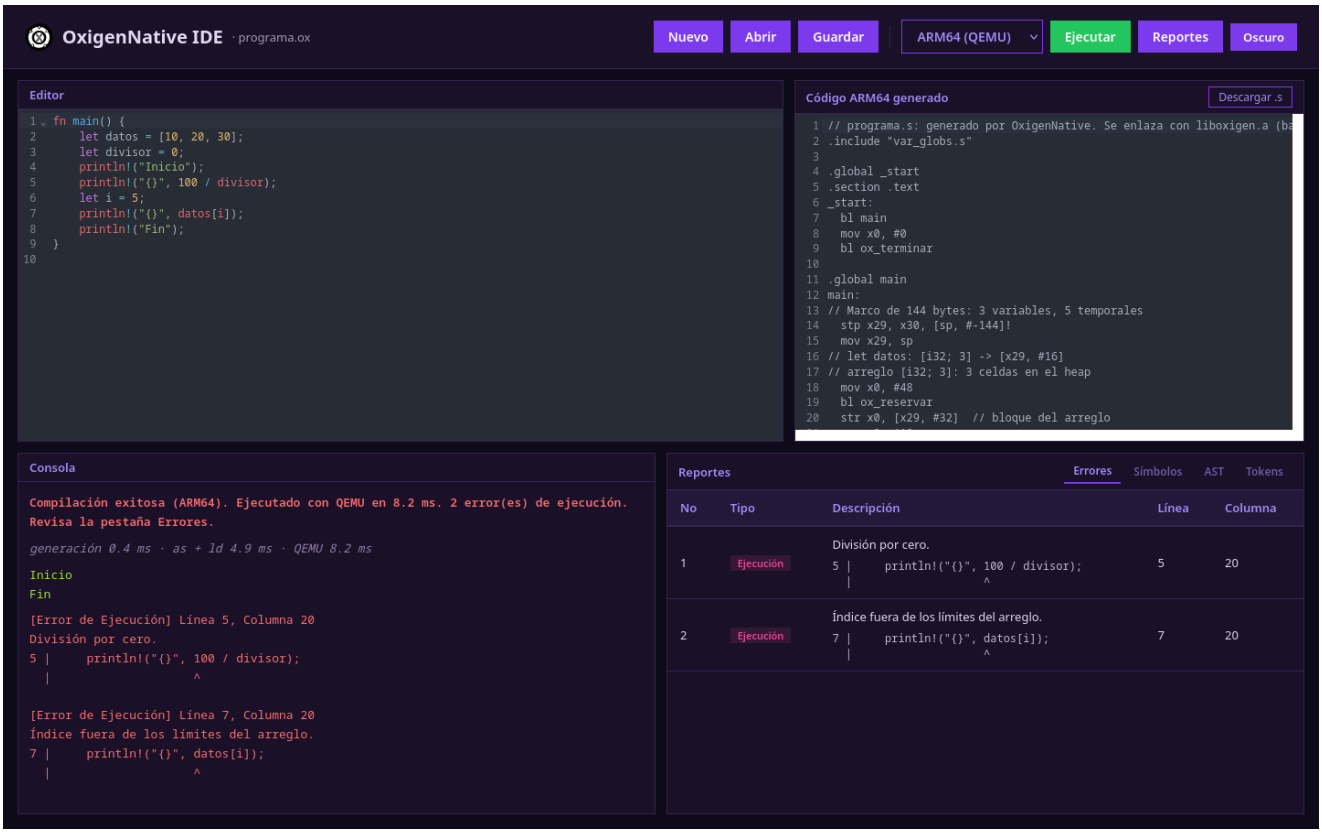


Figura 10: Dos errores de ejecución en ARM64; el programa continúa

El programa se compila y se ejecuta. La división entre cero y el índice fuera de rango solo se detectan al correr, y ninguno detiene el programa: la sentencia que falla se omite y el resto continúa. La salida es `Inicio` y `Fin`, y la tabla muestra los dos errores de ejecución con su posición.

Error semántico en modo ARM64

Archivo `04_inmutable.ox`, el ejemplo de inmutabilidad del enunciado:

Listing 4: 04_inmutable.ox

```
fn main() {
    let edad = 20;

    println!(edad);

    edad = 21;
}
```

Por el error semántico no se genera ARM64. El programa se ejecuta con el intérprete, así que se ve la salida `20` y, después, el error. El aviso naranja del panel ARM64 lo explica (figura del respaldo con el intérprete, en la sección de ejecución).

Manual del Lenguaje

Variables y tipos

OxigenScript es de tipado estático. Las variables se declaran con `let`, que las hace inmutables, o con `let mut` si se desea poder reasignarlas.

Sintaxis

```
let nombre = valor;                 // tipo inferido
let nombre: tipo = valor;           // tipo explícito
let nombre: tipo;                   // sin valor: toma el valor por defecto
let mut nombre = valor;             // variable mutable
let mut nombre: tipo = valor;
```

Tipos de dato

Tipo	Descripción
`i32`	Número entero de 32 bits (ej. `10`)
`f64`	Número decimal (ej. `3.14`)
`bool`	Valor booleano: `true` o `false`
`char`	Carácter (ej. `'A'`)
`String`	Cadena de texto (ej. `String::from("Hola")`)
`[T; N]`	Arreglo de tamaño fijo (ej. `[i32; 5]`), también multidimensional

Valores por defecto

Tipo	Valor por defecto
`i32`	`0`
`f64`	`0.0`
`bool`	`false`
`String`	(cadena vacía)
`char`	`'\0'`
`[T; N]`	Un arreglo con el valor por defecto de `T` en cada celda

Ejemplo

Listing 5: Variables y shadowing

```
fn main() {
    let entero: i32 = 10;
    let decimal: f64 = 3.14;
    let activo: bool = true;
    let contador: i32;          // vale 0
    let mut suma = 0;
    suma = 5;

    // Shadowing: redeclarar una variable con el mismo nombre
    let x: i32 = 10;
    let x: f64 = 3.14;

    println!("{}", entero);
    println!("{}", contador);
    println!("{}", x);
}
```

Operadores de asignación

```
let mut x: i32 = 10;
x = 5;       // asignación simple
x += 3;      // suma compuesta
x -= 2;
x *= 4;
x /= 6;
x %= 5;
```

Operadores

Aritméticos

Operador	Descripción
`+`	Suma de números; concatenación de dos `String`
`-`	Resta
`*`	Multiplicación; repetición de un `String` por un `i32`
`/`	División; entre dos `i32` conserva solo la parte entera
`%`	Módulo; el resultado lleva el signo del divisor
`-x`	Negación unaria

Listing 6: Operadores aritméticos

```
fn main() {
    let a: i32 = 10;
    let b: f64 = 2.5;
    let suma = a + b;                    // 12.5: el i32 se promueve a f64
    let producto = a * 5;                // 50
    let division = 20 / 4;               // 5
    let modulo = 17 % 5;                 // 2
    let negativo = -a;                   // -10
    let texto = String::from("Hola");
    let repetido = texto * 3;            // HolaHolaHola
    println!("{} {} {} {} {} {}", suma, producto, division, modulo,
        negativo, repetido);
}
```

Relacionales

Operador	Descripción
`==` / `!=`	Igual / Diferente
`>` / `<`	Mayor / Menor
`>=` / `<=`	Mayor o igual / Menor o igual

Lógicos

Operador	Descripción
`!`	Negación (NOT)
`&&`	Y lógico (AND)
`||`	O lógico (OR)

En modo ARM64, `&&` y `||` usan corto circuito: si el lado izquierdo ya decide el resultado, el derecho no se evalúa. `&&` tiene más precedencia que `||`, y `*`, `/` y `%` más que `+` y `-`.

Control de flujo

Condicionales

Listing 7: if / else if / else

```
fn main() {
    let x: i32 = 10;
    if x > 5 {
```

```
        println!("Mayor que 5");
    }

    let flag: bool = false;
    if flag {
        println!("Verdadero");
    } else if flag == false {
        println!("Falso");
    } else {
        println!("Otro caso");
    }
}
```

Bucles

Listing 8: while, loop y for

```
fn main() {
    // while
    let mut contador: i32 = 0;
    while contador < 3 {
        println!("{}", contador);
        contador = contador + 1;
    }

    // loop (infinito) con break
    let mut i: i32 = 0;
    loop {
        if i >= 3 { break; }
        println!("Loop {}", i);
        i = i + 1;
    }

    // for sobre un rango (excluye el final) y sobre un arreglo
    for j in 0..3 {
        println!("{}", j);
    }
    for v in [10, 20] {
        println!("{}", v);
    }
}
```

Bucles con etiquetas

Un bucle se puede etiquetar con `'etiqueta`, y `break 'etiqueta` sale de un bucle externo desde uno anidado:

Listing 9: Bucles etiquetados

```
fn main() {
    'externo: loop {
        loop {
            break 'externo;        // sale del bucle externo
        }
```

```
    }
    println!("Fuera de los loops");
}
```

Match

Listing 10: match como sentencia y como expresión

```
fn main() {
    let numero: i32 = 2;
    match numero {
        1 => println!("UNO"),
        2 => println!("DOS"),
        _ => println!("OTRO"),
    }
    let nombre = match numero {
        1 => "uno",
        _ => "varios",
    };
    println!("{}", nombre);
}
```

Funciones

Listing 11: Declaración y llamada de funciones

```
fn saludar(nombre: String) {
    println!("Hola, {}", nombre);
}

fn sumar(a: i32, b: i32) -> i32 {
    return a + b;
}

fn main() {
    saludar(String::from("Ana"));

    let resultado = sumar(10, 5);
    println!("Resultado: {}", resultado);
}
```

Las funciones devuelven un valor con `return expresion;`. Si no declaran tipo de retorno (`->tipo`), son procedimientos. Se admite la recursión. Todo programa debe definir `fn main()`, que es el punto de entrada.

Arreglos

Listing 12: Arreglos

```
fn main() {
    let numeros: [i32; 5] = [10, 20, 30, 40, 50];
    println!("{}", numeros[0]);        // 10
```

```
    println!("{}", numeros.len());      // 5

    let mut matriz = [[1, 2, 3], [4, 5, 6]];
    matriz[1][2] = 7;
    println!("{}", matriz[1][2]);       // 7

    let ceros = [0; 4];                 // repetidor: [0, 0, 0, 0]
    println!("{:?}", ceros);
}
```

Un índice fuera de rango es un error de ejecución: se reporta con su posición y el programa continúa con la siguiente sentencia.

Strings

Listing 13: Strings

```
fn main() {
    let saludo = String::from("Hola Mundo");
    println!("{}", saludo.len());                       // 10
    println!("{}", saludo.contains("Mundo"));           // true
    println!("{}", saludo.replace("Mundo", "Rust")); // Hola Rust
    println!("{}", saludo.to_uppercase());              // HOLA MUNDO
    println!("{}", saludo.to_lowercase());              // hola mundo

    let palabras = saludo.split(" ");                   // arreglo de
        String
    println!("{:?}", palabras);                         // ["Hola", "Mundo
        "]

    let mut texto = String::new();
    texto.push_str("abc");
    println!("{}", texto);                              // abc
}
```

Las **cadenas crudas** (raw strings) escriben el texto sin procesar las secuencias de escape: `rC:\Users"` o `r#"..."#`.

Structs y slices

Los structs y los slices son parte opcional del proyecto y **no se compilan a ARM64**. Un programa que los usa se ejecuta con el intérprete aunque el modo sea ARM64, y el panel ARM64 lo avisa.

Listing 14: Structs y slices (se ejecutan con el intérprete)

```
struct Punto {
    x: i32,
    y: i32,
}

fn main() {
```

```
    let p = Punto { x: 10, y: 20 };
    println!("{}", p.x);                // 10

    let numeros = [10, 20, 30, 40];
    let parte = &numeros[1..3];
    println!("{:?}", parte);            // [20, 30]
}
```

Funciones embebidas

Función	Descripción
`println!("...", args)`	Imprime con los marcadores `{}` o `{:?}` y agrega un salto de línea. Con `println!(a, b)` imprime los valores separados por un espacio.
`typeof(exp)`	Devuelve el tipo de la expresión como texto
`random(a, b)`	Número entero aleatorio entre `a` y `b`
`exp.len()`	Longitud de un string (en caracteres) o de un arreglo
`exp.contains(s)`	Indica si el string contiene la subcadena `s`
`exp.replace(a, b)`	Reemplaza `a` por `b`
`exp.split(s)`	Divide el string y devuelve un arreglo
`exp.to_uppercase()`	Convierte a mayúsculas
`exp.to_lowercase()`	Convierte a minúsculas
`exp.reverse()`	Invierte un arreglo o un string y actualiza la variable
`exp.push_str(s)`	Agrega `s` al final del string

Listing 15: Funciones embebidas

```
fn main() {
    let texto = String::from("Hola Mundo");
    println!("{}", typeof(texto));          // String
    println!("{}", random(1, 10));          // aleatorio entre 1 y 10

    let mut numeros = [10, 20, 30];
    numeros.reverse();
    println!("{:?}", numeros);              // [30, 20, 10]
}
```

Comentarios

Listing 16: Tipos de comentarios

```
// Comentario de una sola línea

/* Comentario
   de bloque */

fn main() {
    let x: i32 = 10;    // comentario al final de la línea
    println!("{}", x);
}
```

Un `/*` sin su `*/` es un error léxico.

Solución de Problemas

Síntoma	Causa y solución
«No se pudo contactar al servidor»	La API no está corriendo. Revise `/tmp/fastapi.log` y vuelva a iniciar con `uv run start.sh`.
El IDE no abre en `localhost:4321`	Node es menor que 22.12 o falta `npm install` en `frontend/`. Revise `/tmp/astro.log`.
«Faltan herramientas: aarch64-linux-gnu-as, . . . »	Instale binutils para AArch64 y QEMU User (sección de instalación).
La pestaña AST no muestra imagen	Falta Graphviz: `dot -V` debe responder.
«Error: .venv no encontrado» al iniciar	Ejecute `uv sync` en la raíz del proyecto.
Puerto ocupado	Otro proceso usa el puerto 8000 o el 4321. Ciérrelo antes de iniciar.
«El programa no terminó en 10 s y se detuvo»	Hay un bucle infinito; QEMU se detiene a los 10 segundos.
«. . . is already defined» al ensamblar	Una función se llama igual que una etiqueta que genera el compilador (`fin_si_1`, `bucle_while_2`. . .). Cámbiele el nombre.
En modo ARM64 `println!` no imprime nada	Una función del usuario empieza con `ox_` (por ejemplo `ox_print`) y reemplaza a la rutina interna del mismo nombre. No use el prefijo `ox_` en funciones propias.

Conclusiones y Recomendaciones

Conclusiones

- OxigenNative compila programas OxigenScript a ensamblador ARM64 y los ejecuta con QEMU, desde el mismo IDE web en que se escriben.
- El panel de código generado permite ver y descargar el `programa.s` de cada ejecución y ensamblarlo a mano.
- Los reportes de errores, símbolos, AST y tokens se actualizan en cada ejecución, y la tabla de errores señala la línea y la columna exactas de cada problema.
- Un programa con errores semánticos, structs o slices se ejecuta con el intérprete, de modo que siempre se ve lo que sí funciona.

Recomendaciones

- Defina siempre la función `main()` como punto de entrada.
- Use `let mut` solo cuando deba reasignar una variable.
- Cuando la salida esté vacía o sea inesperada, revise la consola y la pestaña Errores; un clic en cada fila lleva a la línea del problema.
- No use el prefijo `ox_` en los nombres de sus funciones.

- Guarde sus programas como archivos `.ox` con el botón **Guardar** para poder reutilizarlos.